

SQL - III

Assoc. Prof. Kitsana Waiyamai, Ph. D.
Dept. Of Computer Engineering
Faculty of Engineering, Kasetsart University
Bangkok, Thailand



SQL-II

- Types of **SQL-Join**
- **Sub-queries** in
  - **Select**
  - **From**
  - **Where**
- Etc.

Inner-Join VS Left-Join

```
/*************************************************************
   INNER Join
*************************************************************/
select Student.sID, count(Apply.sID) as number_of_apps
from Student join Apply
on Student.sID = Apply.sID
group by Student.sID;

/*************************************************************
   LEFT Join
*************************************************************/
select Student.sID, count(Apply.sID) as number_of_apps
from Student left join Apply
on Student.sID = Apply.sID
group by Student.sID;
```

COUNT(*) VS COUNT(column)

```
/*************************************************************
 LEFT Join
*************************************************************/
select Student.sID, count(*) as number_of_apps
from Student left join Apply
on Student.sID = Apply.sID
group by Student.sID;

/*************************************************************
  LEFT Join
*************************************************************/
select Student.sID, count(Apply.sID) as number_of_apps
from Student left join Apply
on Student.sID = Apply.sID
group by Student.sID;

/*************************************************************
COUNT(*) counts rows.
COUNT(column) counts non-NULL values in that column.
*************************************************************/
```

SQL- III

- Sub-queries in
  - Where
  - Select
  - From
- CASE
- Common Table Expression (CTE)
- VIEW
- Date & DateTime
- Constraints
- Indexes

SQL: Sub-query in Select

- **Must return exactly one value (scalar)**
  - **NULL if no rows returned**
  - **Error if multiple rows returned**
- **Useful when you need summary next to detail**

sName	GPA	avgGPA
Amy	3.9	3.5666666666666664
Bob	3.6	3.5666666666666664
Craig	3.5	3.5666666666666664
Doris	3.9	3.5666666666666664
Craig	3.4	3.5666666666666664
Edward	2.9	3.5666666666666664

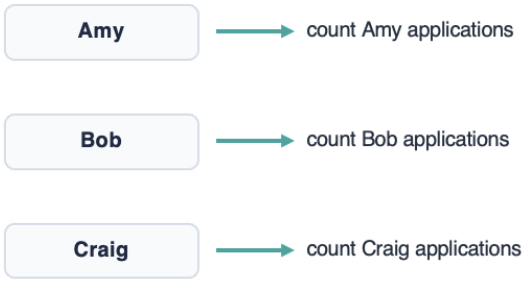
```
/*************************************************************
   Show each student (Name,GPA) with average GPA of all students
*************************************************************/
SELECT sName, GPA, (SELECT AVG(GPA) FROM Student) AS avgGPA
FROM Student;

SELECT sName, GPA,
       round((SELECT AVG(GPA) FROM Student),2) AS avgGPA
FROM Student;
```

SQL: Correlated Sub-query

```
/*****************************************************
 For each student, number of applications
*****************************************************/
select Student.sID, count(Apply.sID) as number_of_apps
from Student left join Apply
on Student.sID = Apply.sID
group by Student.sID;
/*****************************************************
  NOW use sub-query
*****************************************************
select o.sid,
     (select count(*)
     from apply i
    where o.sid = i.sid) as number_of_apps
from student o;
```

Correlated Subquery = the inner query refers to columns from the outer query.



SQL: Sub-query in Select + Where

```
/*****************************************************
 For each student (who applied), show number of applications
*****************************************************/
select Student.sID, count(Apply.sID) as number_of_apps
from Student Inner join Apply
on Student.sID = Apply.sID
group by Student.sID;
/*****************************************************
  NOW use sub-query
*****************************************************
select o.sid,
     (select count(*)
     from apply i
    where o.sid = i.sid) as number_of_apps
from student o
Where o.sid in (select sid from apply);
```

Correlated = the inner query refers to columns from the outer query.

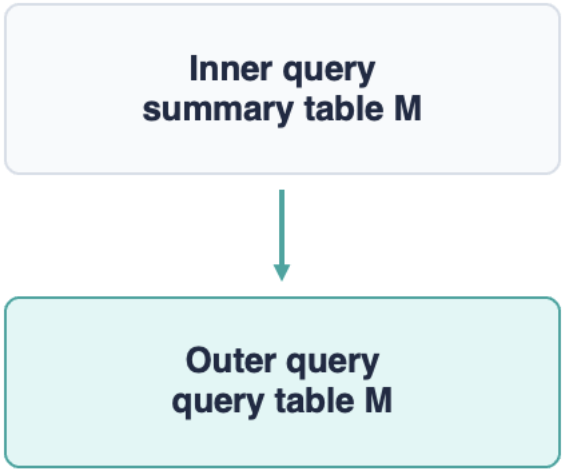
SQL: Sub-query in From

- A subquery must return multiple rows and columns.
- Must have an alias (table name)
- Make complex queries or multi-step logic easier to write.

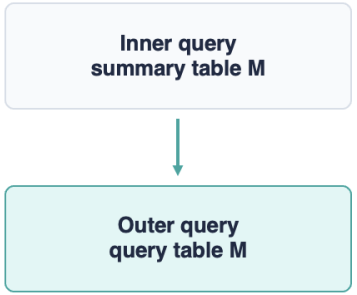
- Widest spread of GPA for each college-major combination

```
select max(mx-mn)
from (select cName, major, min(GPA) as mn, max(GPA) as mx
      from Student, Apply
      where Student.sID = Apply.sID
      group by cName, major) as M;
```

max(mx-mn)
0.8999999999999999



SQL: Sub-query in From



- A subquery must return multiple rows and columns.
- Must have an alias (table name)
- Make complex queries or multi-step logic easier to write

```
/*************************************************************
   Find average GPA by state of colleges that haven been applied to
*************************************************************/
SELECT state, AVG(GPA) as avg_gpa
FROM (SELECT C.state, S.GPA
      FROM Student S, Apply A, College C
      WHERE S.sID = A.sID AND A.cName = C.cName) AS StateGPA
GROUP BY state;
```

state	avg_gpa
CA	3.6999999999999997
NY	3.3666666666666667
MA	3.675000000000003

SQL: Sub-query in Where

- **Purpose:** Filter rows based on conditions
- **Operators:**
  - **Single value:** =, >, <, >=, <=, <>
  - **Multiple values:** IN, NOT IN, ANY, ALL
  - **Existence check:** EXISTS, NOT EXISTS

```
/*************************************************************
   Students with GPA higher than Amy's
*************************************************************/
SELECT sName, GPA
FROM Student
WHERE GPA > (SELECT GPA FROM Student WHERE sName = 'Amy');
/*************************************************************
   Students from largest high school
*************************************************************/
SELECT sName, sizeHS
FROM Student
WHERE sizeHS = (SELECT MAX(sizeHS) FROM Student);
```

SQL: Case

- CASE allows you to implement IF-THEN-ELSE logic within SQL queries.
- SQL's version of if-else statements in programming languages

- CASE is an EXPRESSION, not a statement => **CASE returns a value**

- **CASE Statement Syntax**

```
CASE expression
    WHEN value1 THEN result1
    WHEN value2 THEN result2
    ELSE default_result
END
```

SQL: Case

- Useful for reports, dashboards and business rules that are easy to read.
- CASE returns a value

```
SELECT sName, GPA,
       CASE
           WHEN GPA >= 3.5 THEN 'High'
           WHEN GPA >= 3.0 THEN 'Medium'
           ELSE 'Low'
       END AS GPA_Level
FROM Student;
```

sName	GPA	GPA_Level
Amy	3.9	High
Bob	3.6	High
Craig	3.5	High
Doris	3.9	High
Craig	3.4	Medium
Edward	2.9	Low
Amy	3.9	High
Fay	3.8	High
Jay	2.9	Low
Gary	3.4	Medium
Irene	3.9	High
Helen	3.7	High

SQL: Case + Aggregate

- Useful for reports, dashboards and business rules that are easy to read.
- Convert each row into 1 or 0, SUM the 1/0 values, Get a count per group
- This pattern appears constantly in analytics SQL: **conditional counting and conditional sums**.

major	accepted	rejected
CS	5	2
EE	1	2
biology	1	1
bioengineering	1	1
history	2	1
psychology	1	0
marine biology	0	1

- Number of Rejects or Accepts by Major

```
SELECT major,
       SUM(CASE WHEN decision = 'Y' THEN 1 ELSE 0 END) AS accepted,
       SUM(CASE WHEN decision = 'N' THEN 1 ELSE 0 END) AS rejected
FROM Apply
GROUP BY major;
```

Common Table Expression (CTE)

- **Readable alternative to FROM subqueries.**

- **CTE (**Common Table Expression**)**
  - Reusable intermediate result
  - Temporary result set that you can reference within a SELECT, INSERT, UPDATE, or DELETE statement.
  - Make complex queries easier to write, understand, and maintain by breaking them into simpler parts.

SQL: CTE

```
with CSaccept as (
select sID, cName
from Apply
where major = 'CS' and decision = 'Y')
select * from CSaccept;
```

sID	cName
123	Berkeley
123	Stanford
345	Cornell
987	Berkeley
987	Stanford

```
/************************************************************
CTE is defined within a single query and can only be referenced in that query.
Once the query execution is complete, the CTE is gone
************************************************************/

select Student.sID, sName, GPA
from Student, CSaccept
where Student.sID = CSaccept.sID and cName = 'Stanford' and GPA < 3.8;
```

Error ! Table 'csaccept' doesn't exist

Database View: save a query as a reusable virtual table

- **CTE exists only for one statement.**
  VIEW remains in the database and can be reused.
- A **database view** is a virtual table that represents the result of a query **as a reusable virtual table**
- Unlike a physical table, **a view does not store data itself**; instead, it dynamically retrieves data from one or more underlying tables whenever the view is accessed.
- **View** is not usually a copied table. It is stored query logic.
- **Views** provide a way to simplify complex queries, secure sensitive data, and present data in a specific format without altering the underlying tables.

Defining & Using View

SQL Syntax

```
Create View Vname(A1,A2,…,An) As
<Query>
```

CTE exists only for one statement.
VIEW remains in the database and can be reused.

SQL: View

```
create view CSaccept as
select sID, cName
from Apply
where major = 'CS' and decision = 'Y';

select * from CSaccept;

DROP VIEW CSaccept;

USE VIEW IN QUERY
  Students accepted to CS at Stanford with GPA < 3.8
************************************************************/

select Student.sID, sName, GPA
from Student, CSaccept
where Student.sID = CSaccept.sID and cName = 'Stanford' and GPA < 3.8;
```

SQL: View

```
/*************************************************************
  VIEW LAYERING
  Students accepted to CS at Berkeley with sizeHS > 500
*************************************************************/
create view CSberk as
select Student.sID, sName, GPA
from Student, CSaccept
where Student.sID = CSaccept.sID and cName = 'Berkeley' and sizeHS
> 500;

Use of VIEW

select * from CSberk;
```

SQL: View

```
/*********************************************************
  MAKE ENTIRE JOIN INTO VIEW
  "Universal relation"
*********************************************************/
create view Mega as
select College.cName, state, enrollment,
       Student.sID, sName, GPA, sizeHS, major, decision
from College, Student, Apply
where College.cName = Apply.cName and Student.sID = Apply.sID;

select * from Mega;

/*********************************************************
  NO MORE JOINS IN QUERIES
  High-GPA applications to CS major at large colleges
*********************************************************/

select sID, sName, GPA, cName
from Mega
where GPA > 3.5 and major = 'CS' and enrollment > 15000;
```

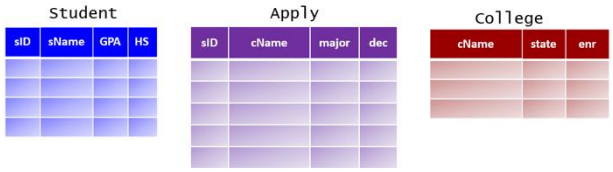
Defining & Using View

Why use views?

- Hide some data from some users
- Make some queries easier / more natural
- Modularity of database access



	s_id	s_Name	gpa	c_name	enrollment
1	543	Craig	3.4	MIT	10000
2	678	Fay	3.8	Stanford	15000
3	765	Jay	2.9	Cornell	21000
4	765	Jay	2.9	Cornell	21000



Real applications tend to use lots and lots (and lots and lots!) of views

SQL-Date and SQL-DateTime

- **Useful MySQL functions**
  - CURRENT_DATE
  - CURRENT_TIMESTAMP
  - YEAR(date), MONTH(date), DAY(date)
  - DATEDIFF(date1, date2)
  - DATE_ADD(date, INTERVAL n DAY)
  - …

SQL Date and DateTime

```sql
CREATE TABLE Employees (
    employee_id INT,
    employee_name VARCHAR(50),
    hire_date DATE);

INSERT INTO Employees (employee_id, employee_name, hire_date)
VALUES (1, 'John Doe', '2023-08-28');


SELECT employee_name, hire_date
FROM Employees
WHERE hire_date = '2023-08-28';


SELECT employee_name
FROM Employees
WHERE hire_date > '2023-01-01';


SELECT MONTH(hire_date) AS hire_month
FROM Employees

SELECT GETDATE() AS CurrentDate;
```

Useful Date functions

- **DATEADD(year, 1, hire_date);**
- **DATEDIFF(day, hire_date, CURRENT_DATE);**
- **Etc.**

Format: `YYYY-MM-DD hh:mm:ss

- **`YYYY`**: Year (4 digits)
- **`MM`**: Month (01-12)
- **`DD`**: Day (01-31)
- **`hh`**: Hours (00-23)
- **`mm`**: Minutes (00-59)
- **`ss`**: Seconds (00-59)

Database Constraints

Constraints are rules enforced by the database

- Protect data quality even when applications, users, or scripts make mistakes.
- Impose restrictions on allowable data, **beyond those imposed by structure and types**.
- Data cleaning starts at insertion time

  - Primary Key (identity + no NULL + unique)
  - Foreign Key (referential integrity, valid reference to another table)
  - Unique (no duplicate values)
  - Non-null (required value)
  - Check (valid range or condition)
  - Other types of constraints

Constraints in Creating Tables

- **not null**
- **primary key** (A_1, ..., A_n)
- **foreign key** (A_m, ..., A_n) **references** *r*

Example*:*

```
create table instructor (
    ID              char(5),
    name            varchar(20) not null,
    dept_name       varchar(20),
    salary          numeric(8,2),
    primary key (ID),
    foreign key (dept_name) references department);
```

Primary key declaration on an attribute automatically ensures **not null**

Constraints in Creating Tables

- **create table** *student* (
  ID **varchar**(5),
  name **varchar**(20) not null,
  dept_name **varchar**(20),
  tot_cred **numeric**(3,0),
  primary key *(ID)*,
  foreign key *(dept_name)* **references** *department*);

- **create table** *takes* (
  ID **varchar**(5),
  course_id **varchar**(8),
  sec_id **varchar**(8),
  semester **varchar**(6),
  year **numeric**(4,0),
  grade **varchar**(2),
  primary key *(ID, course_id, sec_id, semester, year)* ,
  foreign key (*ID*) **references** *student,*
  foreign key (*course_id, sec_id, semester, year*) **references** *section*);

Constraints in Creating Tables

```
/*************************************************************
Make data rules visible in the schema
*************************************************************/
CREATE TABLE Student (
    sID INT PRIMARY KEY,
    sName VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE,
    GPA DECIMAL(3,2) CHECK (GPA BETWEEN 0 AND 4),
    status VARCHAR(10) DEFAULT 'ACTIVE'
);
/*************************************************************
*************************************************************/
CREATE TABLE Apply (
    sID INT,
    cName VARCHAR(50),
    major VARCHAR(50),
    FOREIGN KEY (sID) REFERENCES Student(sID),
    FOREIGN KEY (cName) REFERENCES College(cName)
);
```

Referential Integrity Constraints

Example:

All student IDs and college names in Apply(R) must refer to valid students(S1) and colleges (S2)

```
create table Apply(sID int references Student(sID),
                   cName varchar(15) references College(cName),
                   major varchar(15),
                  decision text);
```

Multi-attribute foreign keys are allowed !!!

Student

sID	sName	GPA	HS

Apply

sID	cName	major	dec

College

cName	state	enr

Referential Integrity Enforcement

Special actions:

- On Delete

  Restrict (default), **Set Null**, **Cascade**

- On Update

  Restrict (default), **Set Null**, **Cascade**

```
create table
Apply(sID int references Student(sID) on delete set null,
      cName varchar(15) references College(cName) on update cascade,
                  major varchar(15), decision varchar(15));
```

Example: University Logical Schema

classroom(*building*, *room_number*, *capacity*)
department(*dept_name*, *building*, *budget*)
course(*course_id*, *title*, *dept_name*, *credits*)
instructor(*ID*, *name*, *dept_name*, *salary*)
section(*course_id*, *sec_id*, *semester*, *year*, *building*, *room_number*, *time_slot_id*)
teaches(*ID*, *course_id*, *sec_id*, *semester*, *year*)
student(*ID*, *name*, *dept_name*, *tot_cred*)
takes(*ID*, *course_id*, *sec_id*, *semester*, *year*, *grade*)
advisor(*s_ID*, *i_ID*)
time_slot(*time_slot_id*, *day*, *start_time*, *end_time*)
prereq(*course_id*, *prereq_id*)

Example: University Logical Schema

```
create table classroom
    (building varchar (15),
    room_number varchar(7),
    capacity numeric (4,0),
    primary key (building, room_number))
```

```
create table department
    (dept_name varchar (20),
    building varchar (15),
    budget numeric (12,2) check (budget > 0),
    primary key (dept_name))
```

```
create table course
    (course_id varchar(8),
    title varchar(50),
    dept_name varchar(20),
    credits numeric (2,0) check (credits > 0),
    primary key (course_id),
    foreign key (dept_name) references department)
```

```
create table instructor
    (ID varchar (5),
     name varchar (20) not null,
     dept_name varchar (20),
     salary numeric (8,2) check (salary > 29000),
     primary key (ID),
     foreign key (dept_name) references department)
```

```
create table section
    (course_id varchar(8),
    sec_id varchar(8),
    semester varchar(6) check (semester in ('Term_1', 'Term_2','Summer')),
    year numeric (4,0) check (year > 2000 and year <2022),
    building varchar (15),
    room_number varchar(7),
    time_slot_id varchar(4),
    primary key (course_id, sec_id, semester, year),
    foreign key (course_id) references course,
    foreign key (building, room_number) references classroom)
```

Indexes

- **Index** helps the database find rows faster
- **Index** work like a book's index, where instead of scanning the entire book (or database table) to find specific information, you can quickly locate it using the index.

- **Index** is a data structure that improves the speed of data retrieval operations on a database table.
- **Persistent data structure**, stored in database
  - Balanced trees (B trees, B+ trees)
  - Hash tables
  - We are focusing on user/application perspective

- **Without Index** scan many rows**, With index** locate rows faster
- **Especially important for Large Tables**.
- **Indexes** are crucial for performance optimization, particularly for queries involving
  - SELECT, JOIN, WHERE, ORDER BY, and GROUP BY clauses.

Indexes

```
Select sName
From Student
Where sID = 18942
```

```
Select sID
From Student
Where sName = 'Mary' And GPA >
   3.9
```

```
Select sName, cName
From Student, Apply
Where Student.sID = Apply.sID
```

Many DBMS's build indexes automatically on **PRIMARY KEY** (and sometimes **UNIQUE**) attributes

Indexes

SQL Syntax

```
Create Index IndexName on T(A)

Create Index IndexName on T(A1,A2,…,An)

Drop Index IndexName
```

Example of Indexes

```
Select * from Employee where
Salary > 5000 and Salary < 7000
```

Id	Name	Salary	Gender
1	Sam	2500	Male
2	Pam	6500	Female
3	John	4500	Male
4	Sara	5500	Female
5	Todd	3100	Male

Table: Employee

Salary	RowAddress
2500	Row Address
3100	Row Address
4500	Row Address
5500	Row Address
6500	Row Address

Index: IX_Employee_Salary

```
CREATE Index IX_Employee_Salary
  ON Employee (SALARY ASC)
```

Indexes

Advantages

- Index = difference between full table scans and immediate location of tuples
  - Orders of magnitude performance difference
- Underlying data structures
  - Balanced trees (B trees, B+ trees)
  - Hash tables

Downsides of Indexes

- Extra space and slower write operations (inserts, updates, deletes).
- Index maintenance.